

1. Why are generics safer than Object?

Generics are safer because they provide type checking at compile time. With `Object`, different types can be stored together, which can cause casting errors at runtime. Generics prevent incompatible types from being added in the first place.

2. When would you create a generic class?

I would create a generic class when I want the same class to work with multiple data types without rewriting the class for each type. The `Box<T>` and `DataManager<T>` classes are examples.

3. Where do you see generics used in Java libraries?

Generics are commonly used in Java collections such as `ArrayList<String>`, `List<Integer>`, `HashMap<String, Double>`, and `Set<Integer>`.

4. Which part was hardest?

The hardest part was understanding how generic methods and generic classes use the type parameter. Once I understood that `T` acts as a placeholder for a specific type, the concepts became easier.

100-Word Synopsis

Generics make Java programs safer, cleaner, and more reusable by allowing classes and methods to work with different data types while maintaining type safety. During this lab, I learned how generics prevent incorrect data from being added to collections and reduce the need for casting. Creating the `Box<T>` class helped me understand how one class can work with Strings, Integers, and Students. The `GenericUtils` methods showed how the same method can work with different lists. Finally, `DataManager<T>` demonstrated how generics can be used to create a reusable data management class. The hardest part was understanding how generic methods use type parameters.